ALBUKHARY INTERNATIONAL UNIVERSITY

DU014 (K)

CRYPTOGRAPHY ESSENTIALS

LITERATURE REVIEW

AES-Based Secure File Encryption and Decryption Tool

Student Name	_____Maralbyek Tilyek_____
Student ID	_____AIU24102280_____
Course / Course Code	Cryptography Essentials CCC2243
Related Deliverable	Project Proposal — AES-Based Secure File Encryption and Decryption Tool
Year / Semester	Year 2, Semester 3, 2025-2026

Table of Content

1. Introduction

Information stored in digital files can be copied, viewed, or modified when an unauthorized person gains access to a computer, removable drive, or shared storage location. File encryption addresses this problem by transforming readable plaintext into ciphertext using a cryptographic algorithm and a secret key. Without the correct key, the ciphertext should be computationally infeasible to understand.

This literature review examines the concepts needed for the proposed AES-Based Secure File Encryption and Decryption Tool. It discusses symmetric cryptography, the Advanced Encryption Standard (AES), authenticated encryption with AES-GCM, password-based key derivation, salts, nonces, file-format design, and secure implementation practices. Each section is written to justify a specific design decision carried forward into the project proposal, and the review concludes by mapping those decisions directly onto the proposal's methodology and evaluation criteria.

2. Symmetric-Key Cryptography

Symmetric-key cryptography uses the same secret key, or related secret key material, for encryption and decryption. The sender and receiver must therefore protect the key and have a secure way to share or establish it. Symmetric algorithms are generally much faster than public-key algorithms, which makes them appropriate for large amounts of data such as documents, images, and videos.

The security of a symmetric system depends on both the algorithm and the secrecy of the key. A strong algorithm cannot protect data if the key is exposed, reused carelessly, or derived from a weak password. For the proposed tool, the user password will not be used directly as an AES key. Instead, a key derivation function will convert the password into a fixed-length cryptographic key while using a random salt to make password-guessing attacks more difficult. This decision is carried directly into Objective 3 and the encryption workflow (Section 6.1) of the project proposal.

3. The Advanced Encryption Standard (AES)

AES is a block cipher standardized by the United States National Institute of Standards and Technology (NIST) in FIPS PUB 197. It operates on 128-bit blocks and supports key lengths of 128, 192, and 256 bits. AES replaced the older Data Encryption Standard because DES used a comparatively short 56-bit key and became vulnerable to exhaustive key-search attacks as computing power increased.

AES itself is a primitive that encrypts one fixed-size block at a time. To encrypt a complete file, AES must be used through an operation mode. The mode determines how multiple blocks are processed and whether additional security properties, such as authentication, are provided. Therefore, selecting AES alone is not sufficient; the implementation must also select a secure mode, generate the required random values correctly, and handle keys and errors safely. This is precisely why the proposal specifies not just "AES" but the complete construction: AES-256-GCM.

4. AES-128 and AES-256

AES-128 and AES-256 use the same 128-bit block size but different key lengths. AES-256 has a larger key space and provides a higher security margin against exhaustive search. AES-128 is already considered secure for many applications, while AES-256 is a reasonable choice for a student project that aims to demonstrate strong protection and future-oriented security. AES-256 is the variant adopted in the project proposal (Section 7, Tools and Technologies).

The larger key does not automatically compensate for poor implementation. A weak password, predictable nonce, reused nonce, or incorrect file handling can undermine either AES-128 or AES-256. For this reason, the proposed tool will prioritize a well-reviewed cryptographic library — the Python cryptography package, as specified in the proposal — over implementing AES mathematical operations manually.

5. Authenticated Encryption and AES-GCM

Confidentiality hides the contents of a file, but confidentiality alone does not prove that the encrypted file has not been modified. An attacker might alter ciphertext, damage it accidentally, or replace it with another file. A secure file-encryption tool should therefore provide both confidentiality and integrity.

Galois/Counter Mode (GCM) is an authenticated-encryption mode for block ciphers. AES-GCM encrypts the plaintext and produces a 128-bit (16-byte) authentication tag. During decryption, the tag is checked before the plaintext is accepted. If the password is wrong or the ciphertext, nonce, associated data, or tag has been altered, authentication fails and the application must reject the result rather than writing corrupted or unauthenticated plaintext. This is the exact behavior specified in the proposal's decryption workflow (Section 6.2, Step 3) and reflected in the "Security" row of its Testing and Evaluation Criteria table.

A critical AES-GCM requirement is that a nonce must never be reused with the same key. The application should generate a fresh, unpredictable 96-bit (12-byte) nonce for every encryption operation — the exact size specified in the proposal's methodology. The nonce does not need to be secret.The proposed system will use a cryptographically secure random nonce and will treat nonce reuse as a serious implementation error, consistent with the risk mitigation described in the proposal's threat model (Section 6.3).

6. Passwords, Keys, and PBKDF2

Users commonly remember passwords rather than random binary encryption keys. Passwords are usually shorter and less random than keys, making them vulnerable to dictionary and brute-force attacks. A password-based key derivation function (KDF) increases the cost of guessing by applying a pseudorandom function repeatedly and producing a key of the required length.

PBKDF2 is a standardized password-based key derivation method described in RFC 8018. It combines a password, a salt, an iteration count, and a pseudorandom function such as HMAC-SHA-256. The salt is

public random data — a cryptographically random 16 bytes in the proposed design — that ensures the same password produces different derived keys in different encryption operations. It also prevents an attacker from efficiently reusing precomputed password tables across many files.

The iteration count controls the computational cost of deriving a key. Following current OWASP password-storage guidance adapted for key derivation, the proposed application will use a minimum of 480,000 iterations of PBKDF2-HMAC-SHA256, the exact figure specified in the project proposal's methodology and tools sections. This value should still be validated on the target development machine to confirm that password guessing is made expensive while ordinary encryption and decryption remain fast enough for interactive use. The exact iteration count is recorded as part of the encrypted file's header so that decryption can reproduce the same derivation process. PBKDF2 does not make a weak password strong; it only makes large-scale guessing more expensive. The application should therefore encourage — though, consistent with the proposal's scope, not enforce — long, unique passwords.

7. Salt, Nonce, Authentication Tag, and File Format

An encrypted file must preserve several values needed for decryption. The salt is required to derive the same key from the password. The nonce is required by AES-GCM to reproduce the encryption context. The authentication tag is required to verify that the ciphertext has not been changed. These values are not secret, but they must be stored accurately and parsed safely.

A practical output format can begin with a small versioned header followed by the salt, nonce, authentication tag, and ciphertext, matching the binary header structure defined in the project proposal (Section 6.1, Step 5). A version field allows the application to change its format in the future without confusing old and new files. The original filename may also be stored, but the application should avoid exposing unnecessary sensitive metadata. The output should use a dedicated extension such as .enc, consistent with the proposal's example output naming (document.pdf.enc), so that users can distinguish encrypted files from ordinary files.

The application should decrypt into a temporary file and replace or create the destination only after AES-GCM authentication succeeds. This prevents an incorrect password or tampered ciphertext from leaving a misleading partial output, and directly implements the mitigation described against interrupted or incomplete writes in the proposal's threat model (Section 6.3).

8. Secure Software Implementation

Cryptographic software is difficult to implement safely because small mistakes can invalidate otherwise strong mathematics. The proposed project should use a mature library such as Python cryptography rather than implementing AES, GCM, or PBKDF2 from scratch. Library APIs should be used exactly as documented, and exceptions should be handled without exposing passwords, keys, or sensitive plaintext in messages or logs.

Passwords should be collected without displaying their characters and should not be stored after the operation is complete. Keys and plaintext should remain in memory only for as long as necessary. The

interface should provide general error messages, such as "decryption failed," instead of revealing whether the password was almost correct or whether a particular part of the encrypted file was valid. These practices reduce information leakage and align with the non-technical, clear-messaging requirement set out in Objective 5 of the project proposal.

9. Related Applications and Security Considerations

Existing file-encryption applications demonstrate that users need more than an encryption button. They need clear file selection, understandable status messages, protection against accidental overwriting, and reliable recovery of files. Some applications integrate encryption with operating-system access controls, cloud synchronization, key management, or password managers. A small academic tool will not reproduce all of those features, and the proposal's scope explicitly excludes cloud integration, multi-user access control, and enterprise key management — but it can demonstrate the core cryptographic workflow correctly.

The main risks for this project are weak passwords, lost passwords, insecure key storage, nonce reuse, accidental deletion of the original file, and failure to verify authentication before releasing decrypted data. These risks correspond directly to the three risks identified in the proposal's threat model (Section 6.3): weak/reused passwords, nonce reuse, and interrupted writes. Encryption does not recover a forgotten password, and it does not protect a file after it has been successfully decrypted. The project documentation should state these limitations clearly, consistent with the "password recovery" exclusion listed under the proposal's Out of Scope section.

10. Comparison of Design Options

Option	Strengths	Limitations / Decision
AES-GCM	Provides encryption and tamper detection in one standard mode.	Requires strict nonce uniqueness. Selected for this project.
AES-CBC with separate MAC	Well-known confidentiality mode; can be combined with HMAC.	More complex to implement correctly because encryption and authentication must be combined safely. Not selected.
AES-128	Strong security and slightly lower computational cost.	Smaller security margin than AES-256. Not selected for the main prototype.
AES-256	Large key space and strong long-term security margin.	Requires correct password-based key derivation; selected for the project.
Direct password use	Simple to explain and code.	Unsafe because human passwords are not uniformly random keys. Rejected.
PBKDF2-derived key (480,000 iterations)	Standardized, available in trusted libraries, and slows guessing.	Still depends on password quality and a suitable iteration count. Selected.

11. Relevance to the Proposed Project

The reviewed research directly supports the proposed architecture. Python will provide the application environment, Tkinter will provide the graphical interface, and the cryptography package will provide tested implementations of AES-GCM and PBKDF2, exactly as listed in the proposal's Tools and Technologies table. The application will generate a fresh 16-byte salt and 12-byte nonce for each encryption operation, derive a 256-bit key from the user password using PBKDF2-HMAC-SHA256 at 480,000 iterations, store the non-secret parameters with the encrypted data, and verify the authentication tag before creating a decrypted file.

The literature also establishes the evaluation criteria for the project, which map directly onto the five categories defined in the proposal's Testing and Evaluation Criteria table. Correctness is assessed through successful round-trip encryption and decryption with SHA-256 hash comparison; Security is assessed through rejection of incorrect passwords and rejection of modified encrypted files; Robustness is assessed through preservation of binary file contents and correct behavior with empty and very large files; Usability is assessed through protection against accidental overwriting and clear error messaging; and Performance is assessed against the proposal's stated target of under 5 seconds for a 100 MB file, to be measured with the pytest test suite specified in the proposal's tools. These tests will assess both functionality and the security properties claimed by the application.

12. Conclusion

AES is an appropriate foundation for a local file-encryption tool because it is standardized, efficient, and widely supported. The review shows that secure file encryption requires more than choosing a strong algorithm: it requires an authenticated mode, unique nonces, password-based key derivation, random salts, careful file-format design, and safe error handling. AES-256-GCM with a PBKDF2-HMAC-SHA256-derived key at 480,000 iterations is therefore a suitable design for the proposed project, provided that a trusted cryptographic library is used and the limitations of password-based protection — as already acknowledged in the project proposal's scope and threat model — are documented clearly in the final report.

References

Dworkin, M. (2007). *Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC* (NIST Special Publication 800-38D). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-38D

Kaliski, B. (2017). *PKCS #5: Password-based cryptography specification version 2.1* (RFC 8018). Internet Engineering Task Force. https://doi.org/10.17487/RFC8018

National Institute of Standards and Technology. (2001). *Advanced Encryption Standard (AES)* (FIPS PUB 197). U.S. Department of Commerce. https://doi.org/10.6028/NIST.FIPS.197

National Institute of Standards and Technology. (2010). *Recommendation for password-based key derivation: Part 1, storage applications* (NIST Special Publication 800-132). U.S. Department of Commerce. https://doi.org/10.6028/NIST.SP.800-132

National Institute of Standards and Technology. (2015). *Recommendation for key management, Part 1: General* (NIST Special Publication 800-57 Part 1 Revision 4). U.S. Department of Commerce. https://doi.org/10.6028/NIST.SP.800-57pt1r4

OWASP Foundation. (n.d.). *Cryptographic storage cheat sheet*. OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/Cryptographic_Storage_Cheat_Sheet.html

OWASP Foundation. (n.d.). *Password storage cheat sheet*. OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

Python Cryptographic Authority. (n.d.). *Symmetric encryption and authenticated encryption documentation*. cryptography. https://cryptography.io/en/latest/hazmat/primitives/aead/